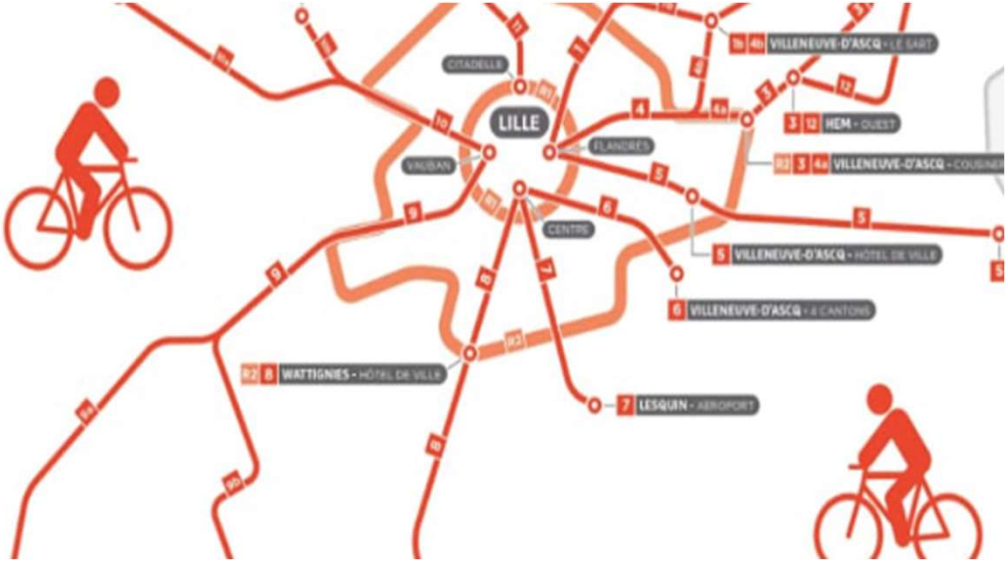
VéloVille Hauts-de-France — Valorisation et Fiabilisation des Données

M2I
RNCP 39103 [Bloc2]

Kheira Bekara
20 Octobre 2026

Plan

1. **Présentation du projet**
2. **Contexte Métiers & Périmètre**
3. **Audit des Données, Fiabilisation ETL & Idempotence**
4. **Architecture Technique & Modèle de Données MLD**
5. **Sécurité, Authentification JWT & Matrice des Droits**
6. **REST API & Restitution Front-End Streamlit**
7. **Tests Unitaires , d'Intégration & couverture**
8. **Démarche de Recette & Gestion des Anomalies**
9. **Choix Écoresponsables, Versioning & Maintenance**
10. **Bilan, Limites & Axes d'Amélioration**





Contexte Vélo-Ville

Besoin : Valoriser les données hétérogènes de 120 stations, 1 800 vélos et ~9 000 trajets/jour

Périmètre livré

- Ingestion et fiabilisation des 3 ressources
  - Stations
  - Trajets
  - Incidents
- API Restful sécurisée (JWT)
- Restitution tabulaire Streamlit (2 profils)

Périmètre reporté

- Données météo
- Modules d'anticipation/prédiction

Audit données & Ingestion ETL

Audit des 2 330 lignes brutes : Identification des doublons, types invalides, valeurs négatives/nulles

Stratégie de Fiabilisation

- Fonctions pures sans BDD (RG1 à RG8):
  - *Normalisation* : Correction automatique de la casse, suppression d'espaces superflus, formats ISO 8601
  - *Rejet strict (RG1 à RG8): isolation des durées <=0, stations d'origine inconnues ou incohérences chronologiques*

Garantie ETL & Idempotence

- Production d'un journal des rejets et d'un journal d'avertissements
- Idempotence: Re jouabilité infinie du script (importer.py) sans doublons , ni modification des comptages

Architecture Technique & Modèle MLD

architecture 3-Tiers

- **BDD: PostgrSQL 14+**
  - **Stations/ trajets/ incidents/ utilisateurs**
- *Back-End: FastAPI + SQL Alchemy +Pydantic*
- *Front-End : Streamlit responsive*

Garanties du schéma relationnel

- Clés étrangères vers ***Station*** obligatoires pr les départs de trajets et stations visées par un incident
- Contraintes d'intégrité 'durée >0, statut restreint à ***ouvert, en_cours, resolu***)

Sécurité, Authentification & Matrice des droits

Authentification & Sécurité des

- Mots de passe hachés avec bcrypt dans PostgreSQL
- Jeton JWT (HS256) à durée d'expiration limitée
- Protection des variables sensibles via .env

Contrôle d'accès basé sur les rôles

- Direction: Lecture seule sur l'ensemble des ressources et indicateurs
- Exploitant: Lecture + Droit exclusif de modification du statut des incidents
- Gestion HTTP: si token absent/expiré (401), si rôle insuffisant (403)

API REST & Restitution Front-End

API Restful

- Endpoints d'agrégation SQL performants
  - /indicateurs/activite-par-jour
  - /stations-actives
  - /incidents-par-statut
- Endpoints paginés et filtrables

Interface Streamlit

- Connexion (stockage et sécurisé en st.session_state)
- Ecrans « Stations », « Incidents », et « Indicateurs » (st.metric et st.dataframe)

Tests & couverture

Stratégie de Test

- Suite de tests automatisée **:** 31 tests couvrant l'ensemble de la chaîne de traitement (Fiabilisation, API, Sécurité & Indicateurs
- Exécution rapide (31 PASSED)

Détail des couvertures

- Tests Unitaires (23 tests)
  - *Règles de Gestion & ETL (19 tests)* : validation de la propreté des dates, gestion des valeurs nulles/négatives, détection des doublons et validation des statuts d'incidents
  - *Métriques & Indicateurs (4 tests)* : vérification des agrégations métier
- Tests d'Intégration API & RBAC (8 tests)
  - *Authentification JWT* : réponses HTTP conformes
  - *Matrice des Droits (RBAC)* : vérification stricte du rôle lors de la modification des incidents

Démarche Recettes & Gestion des anomalies

Campagne de recette 15/10/2026	Anomalies et Traçabilité	Décision du procès-Verbal
➢ 10 cas de tests fonctionnels joués sur une BDD réinitialisées ➢ Couverture : Ingestion ETL idempotente, sécurité JWT, contrôle d'accès, pagination	➢ 4 anomalies relevées lors de l'exécution initiale ➢ 2 anomalies critiques corrigées avec commits dédiées et des tests de non-regression (anomalies 1 & 2) ➢ 2 réserves mineurs reportées (anomalies 3 & 4)	➢ Recette PRONONC2E AVEC RESERVES: L'application est conforme aux exigences métiers et fonctionnelles du CdC

Choix Ecoresponsables & Versionning

Sobriété Numérique

- Pagination côté API pour limiter les transferts réseau inutiles.
- Requêtes SQL ciblées évitant les surcharges mémoire.
- Pas de rafraîchissement temps réel : Chargement à la demande
- Application Streamlit légère sans composants lourds superfétatoires.

Traçabilité Git

- Commits atomiques structurés par briques fonctionnelles.
- Tag de livraison officiel : **v1.0.0-rc1**
- Hash de commit référencé : **e73320c**

Bilan & Perspectives

Objectifs atteints & limites

- API fonctionnelle, fiabilisation à 100%, interface Streamlit fluide et sécurisée
  - API documentée
- Limites
  - Gestion des utilisateurs statiques via script SQL
  - Pas de représentation graphique

Axes d'amélioration

- Module d'ingestion des données météo et modèles de prédictions de rééquilibrage de la flotte
- Interface de gestion automatiques des comptes et des rôles

Conclusion

Nous proposons pour **l'API VéloVille** une conception

- **Standard** (FastAPI, REST, OpenAPI/Swagger intégré
- **Sécurisée** (Authentification JWT et cloisonnement par rôles)
- **Performante** (Agrégations SQL directes et pagination côté serveur)
- **Testable**

Merci pour votre attention !